Summary:

This thesis investigates Android malware detection using opcode-based features and machine learning techniques. Employing opcode n-grams and a neural network model, the study achieves effective classification of Android applications into benign and malicious categories. With a feature dimension of 2048 and n-gram lengths of 3, the model demonstrates robust performance validated through cross-validation, showcasing consistent differentiation capabilities across datasets. Despite challenges such as limited dataset size and occasional validation fluctuations, the method proves promising for static analysis.

Contents

1. Introduction 3
2. Overview 4
2.1 What is malware? 4
2.2 Malware analysis 6
2.3 Android environment and security 7
2.4 Machine learning and malware detection 8
3. Approach and Techniques 10
3.1 Opcode extraction 10
3.2 Data preprocessing 11
3.3 Dataset 12
3.4 Neural networks 12
3.5 Used metrics 13
3.6 Data visualization 14
4. Implementation 16
4.1 Goal of the project 16
4.2 APK decompression 16
4.3 Feature extraction 17
4.4 Model structure 19
5. Test of solution 22
5.1 Model training 22
5.2 Model deployment 28
6. Conclusions 30
7. Bibliography 31
8. List of figures and tables 32

1. Introduction

The topic of security and malware detection is crucial in today's digital age due to the increasing number of cyber threats targeting individuals and organizations. As cyber-attacks become more sophisticated, the need for effective malware detection mechanisms is more pressing than ever. Effective security measures not only protect sensitive data but also ensure the integrity and availability of critical systems.[1]

The objective of this thesis is to design a malware detection solution based on opcode frequencies in apk files using machine learning techniques, specifically, a neural network. These opcode sequences will be extracted using reverse engineering those files and further hashed for dimensionality reduction. For the purpose of most optimal performance multiple model and parameter structures will be tested and the best one will be deployed.

The first chapter gives an overview on the cyber security landscape, particularly for the android systems. Second chapter explains the application and reasoning behind the methods and metrics used for the project. Third chapter gives a detailed overview on the practical implementation of the solution. The following chapter describes the experimentation process and showcases its results. The final chapter discusses the overall outcomes and implications.

[1] ENISA, *ENISA THREAT LANDSCAPE,* October 2023

2. Overview

2.1 What is malware?

Malware, short for malicious software, is a generic term for programs created with malevolent intent. Such programs can be created from scratch or they can represent an altered version of genuine software designed to trick victims. Malware can be grouped into three generalized categories:[2]

- Viruses - a type of malicious code that replicates by embedding itself into other programs, known as its hosts. For a virus to function and cause harm, it requires an existing host program. For instance, a virus might attach itself to a software utility, such as a word processing application. When the application is launched, the virus is activated, potentially replicating itself and disabling malware detectors on the computer system.
- Worms - it replicates itself by executing its own code without needing a host program, unlike a virus. The key difference between viruses and worms is that worms do not require a host to cause harm. Additionally, viruses typically spread through programs or files on a single computer, whereas worms propagate through network connections, aiming to infect as many connected systems as possible.
- Trojans - embedded in an application or system by its designer. It appears to perform a useful function, like displaying local weather, but secretly performs unauthorized actions, such as capturing and sending the user's keystrokes to a malicious host. Trojan horses are often used to access and transmit unauthorized information and can also be classified as spyware. Additionally, embedded malware can perform other malicious activities, such as acting as a time bomb that triggers harmful actions at a specific time, like refusing all service requests on a particular date.

The history of malware is extensive, encompassing a vast array of types and instances that could easily warrant a dissertation on their own. However, for the purposes of this summary, we will rely on a timeline provided by IBM[3] to highlight some milestones in the evolution of malware.

1966 - John von Neumann developed a concept of a self-replicating program.

1971 - Bob Thomas created an experimental program called *Creeper* designed to be able to move itself between different computers. It was further modified by Ray Tomlinson to not only move itself but to make copies of itself.

1982 - Rich Skrenta creates a program called *Elk Cloner*. Intended as a joke the program could replicate itself via floppy disks and infect Apple computers. Considered to be first malicious program due to its tendency to overwrite and delete data.

---

[2] N. Idika, A. Mathur, *A survey of malware detection techniques,* Purude Univeristy 2007, p. 5
[3] https://www.ibm.com/blog/malware-history/

1986 - *Brain virus* created by Amjad and Basit Farooq Alvi was the first IBM Personal Computer virus and the first one to spread worldwide. It was designed to be an anti-piracy measure which upon booting up illegitimately copied software would prompt user to call the creators to receive a vaccination. Brothers located in Pakistan received many calls around from around the globe indicating just how widespread the copyright violation was.

1988 - A malware precursor called *Morris worm* had been created by Robert Morris, Initially created as a proof-of-concept rather than with malicious intent it was designed to exploit a backdoor in Unix system and stay hidden. The worm quickly spread around the internet. Since the worm would not only copy itself on different computer it would also repeatedly replicate itself on infected computers eating up memory. This caused a lot of damages and it considered to be the first cyberattack.

2000 - Onel de Guzman created the *ILOVEYOU virus*. Considered to be the first significant piece of outright malware it was created to steal passwords via email attachments disguised as love letters. Besides stealing passwords the worm also deleted files and caused millions in damages.

2013 - *CryptoLocker* - one of the first instances of ransomware, a type of software created to extort victims by withholding access to the computer, was created. It would encrypt files and keep them unusable until the victim paid 2 bitcoins, or roughly $175 at the time.

2016 - With the rise of internet of things, smart IoT devices present new vulnerabilities. Created by Paras Jha the *Mirai botnet* took over a massive number of CCTV cameras with weak security. Originally designed to target gaming servers with denial of service attacks the bot net was more powerful than the author anticipated and it even managed to attack a major DNS provider cutting off a big part of eastern seaboard in the United States from the internet for a day.

2017 - State-sponsored cyberattacks and virtual espionage surged, marked by two significant malware incidents. The Petya ransomware, initially spread through phishing, was modified into the more destructive NotPetya wiper worm, which destroyed data even if ransom was paid. This attack is believed to be tied to Russian intelligence targeting Ukraine. The same year, the WannaCry ransomware worm affected high-profile targets, notably Britain's National Health Service, and is suspected to be linked to North Korean government sectors. Both attacks exploited a Microsoft Windows vulnerability called Eternalblue, discovered by the NSA, which faced criticism for not reporting it before hackers exploited it.

2019 - In recent years, ransomware has seen fluctuations in activity. Although the number of successful attacks may be declining, hackers are now targeting more high-profile victims and causing greater damage. A concerning trend is the rise of Ransomware-as-a-Service (RaaS), available on dark web marketplaces. RaaS allows professional hackers to carry out ransomware attacks for a fee, enabling even those without advanced technical skills to launch attacks if they have the financial means.

2.2 Malware analysis

Understanding different types of malware and their evolution is crucial for detecting and mitigating these threats. To detect malware, the program needs to be analyzed for any potential clues to bring closure as to its nature and goal. Various methods are employed in this task, classified based on how the information is extracted and the environment setup during testing.

Based on information received there are:[4]

- Anomaly-based analysis – used to detect deviations from usual behavior of the system, which can indicate malicious activity. This involves two phases. The first learning phase aims to learn the normal behavior and to establish a baseline for comparison. After that a monitoring phase is executed to monitor any deviations from the baseline. The method is particularly useful at detecting zero-day attacks. This, however, comes at a cost of high false positive rate and a need to meticulously select particular features used in detection.
- Specification-based analysis – a derivative of anomaly based analysis that aims to correct the high false positive rate. Instead of approximating the implementation of an application or system, specification-based detection approximates the requirements for an application or system. During the training phase, it establishes a rule set that specifies all valid behaviors for the system being protected or the program under inspection. Similarly to previous method there is a need to determine appropriate features. This features are more varied and complex as well and it can be difficult to express these features in a form amenable for a machine.
- Signature-based analysis - also known as pattern matching, is a method used to identify and detect known malware based on predefined signatures or patterns. This approach relies on comparing the characteristics of files, code, or network traffic against a database of known malware signatures. In essence the program under investigation is tested to see if it contains any malware indicating signatures. Its main benefit is low false positive rate. The problem with this method is a need for that extensive database and general inability to adapt to unknown software.

And based on experimentation environment there are:[5]

- Static analysis – performed without executing the program. Focuses on inspecting the code and structure of the software. This usually entails extracting metadata like file headers or code snippets. One of useful methods in this type analysis is calculation of file's hash for quick identification of any malicious files contained in a database for comparison. Due to not running any code is method is the safest. It's also relatively quick and works exceptionally when dealing with known malware. However, it is less effective

[4] N. Idika, A. Mathur, op.cit., p. 9, 18, 31-33

[5] R. Sihwail, K. Omar, K. A. Z. Ariffin, *A Survey on Malware Analysis Techniques: Static, Dynamic, Hybrid and Memory Analysis*, International Journal on Advanced Science Engineering and Information Technology 8(4-2):1662, 2018

against unknown malware. To counter static analysis, malware authors often use evasion techniques such as obfuscation or encryption.

- Dynamic analysis - performed during program execution. This should be done in a safe environment i.e. using a virtual machine without internet connection. Creating an isolated environment ensures no harm to the actual hardware and data is done. Program executing provides practical knowledge as to the functionality of tested malware. This is performed by monitoring different metrics like system or API calls, network traffic and memory usage. Using this method we can monitor processes that encompass the malware and all modifications to the system can be seen.
- Hybrid analysis – integrates static and dynamic analysis. Usually static analysis is done first and followed by the dynamic one for a more comprehensive understating of malware. This approach combines the strengths of both methods, enhancing the ability to detect and analyze sophisticated malware.

In practice these approaches are used concomitantly to ensure as robust and adaptable system for malware detection.

2.3 Android environment and security

The rapid evolution of handheld devices, particularly smartphones and tablets, has revolutionized how we connect, communicate, and conduct daily activities. At the forefront of this technological shift is the Android operating system, developed by Google and embraced by a vast array of manufacturers worldwide.

Android devices face a diverse range of threats, including malware, phishing attacks, data breaches and vulnerabilities in both system components and third-party applications. The dynamic nature of the Android ecosystem makes it a prime target for cybercriminals seeking to exploit vulnerabilities and proves a challenge for Google to fight malicious activity.

Google Security Services, such as Google Play Protect and Play Integrity API, help detect malware and device compromises on GMS-certified devices. Play Protect scans apps for threats, and the App Defense Alliance collaborates with industry partners to enhance security. Enterprise admins can use allow lists and block lists on the Google Play Store to control which apps are allowed, reducing malware risks. The Play Integrity API enables developers to verify that interactions come from genuine app binaries on authentic Android devices, preventing fraud and tampering. Enterprise Mobility Management (EMM) partners can use these services to enforce policies restricting app installations to trusted stores like Google Play, and receive signals to detect and mitigate compromises.[6]

Google Play Protect comes pre-installed in all devices that have Google Play and protects more than 3.5 billion devices. It performs regular scans once a day for any unwanted behavior. When detecting a suspicious app it alerts the user and automatically removes or quarantines malicious

[6] Android Security Paper 2023 p.44

software. It also enables user to send any unknown apps to Google for further analysis. Google Play Protect's install verification checks apps coming from outside of the Play Store when users try to install it. If the app is known to be malware, Google Play Protect will stop the installation and prevent it from harming users. To further enhance as of recently the protection install verification performs real-time threat detection for new apps from sources outside Google Play Store, to help fight emerging threats.[7]

Another security measure is application signing. Android apps are required to be signed with a developer key prior to installation. It helps to ensure the integrity and authenticity of software. Developers need to generate a keystore file containing private keys and certificates. These keys are used to sign the app. During the build process, the APK (Android Package) file is signed with the developer's private key. This adds a digital signature to the APK. When a user installs the app, the Android system verifies the signature against the public key in the certificate to ensure the app's integrity. It's crucial to protect the keystore file and its passwords, as compromising these can lead to unauthorized app modifications or updates.[8] It can be said that it works similarly to signature-based detection but instead of detecting malware it ensures application authenticity.

In addition, Android employs mandatory encryption to safeguard user data if a device is lost or stolen. It utilizes file-based encryption (FBE), which allows different directories to be encrypted with separate keys. With FBE, the device boots directly to the lock screen and becomes fully usable almost immediately after unlocking. There are two types of storage locations: Device Encrypted (DE) storage, available once the device boots and before the user unlocks it, protected by a hardware secret and software in the Trusted Execution Environment (TEE) that verifies the boot process; and Credential Encrypted (CE) storage, accessible only after the user has unlocked the device, providing additional protection against brute force attacks in hardware. Most apps store data in CE storage and run only after credentials are entered. On multi-user devices, each user has unique CE and DE keys, protected by their lock screen credentials. Encryption keys are 256 bits long and generated randomly on-device.[9] This makes it near impossible to extract data directly from a drive and ensures safety even if someone gains physical access to the device as long as they cannot authorize it.

2.4 Machine learning and malware detection

In the former section it's stated how described analysis methods struggle with adaptability to new threats. Traditionally most of these operations were performed manually with researchers choosing appropriate parameters to monitor and adjusting detection algorithms to reflect them. With the use of machine learning these processes can be automated ensuring a higher degree of adaptation to ever evolving dangers. Furthermore, as stated in the previous section, we outlined how Google Play Protect scans files in real-time, especially considering open and rapidly

[7] Ibidem, p.46-47
[8] Ibidem, p.45
[9] Ibidem, p.49

evolving development environment; a machine learning approach in machine learning might be beneficial. In fact Play Protect already utilizes machine learning.[10]

There are multiple approaches to using machine learning in malware detection with respect to different types of analysis techniques but the one this thesis focuses on is signature-based and thus will be the only one to be expanded upon.

Signature in this sense is a feature encapsulating program structure helping in identifying unique malware. Signature-based detection approach is widely used within commercial antivirus. One of the ways to get these features is to extract operation code (opcode) sequences. These sequences are then compared among different samples by a machine learning algorithm. Captured differences between tested files can work as signatures to separate benign and malicious software. [11]

The approach is based on guidelines outlined in a referenced book.[12] Detailed descriptions and implementations of these methods are provided in the subsequent sections.

[10] https://developers.google.com/android/play-protect

[11] Ö.Aslan, R.Samet, *A Comprehensive Review on Malware Detection Approaches*, IEEE Access, January 2020, p.6252-6253

[12] J. Saxe, H. Sanders, *Malware Data Science: Attack Detection and Attribution*, No Starch Press 2018, p.199-215

3. Approach and Techniques

3.1 Opcode extraction

Opcode, or operation code, is a form of machine language instruction that specifies the operation to be performed. In essence, opcodes are a set of commands mapped to specific hexadecimal values, which are collected in an opcode table for quick lookup. This table serves a function similar to that of mnemonics, providing a quick way to translate human written code to machine language and vice versa, essentially allowing for efficient mapping of high-level programming constructs to low-level machine instructions.[13]

Opcode	Mnemonic	Description
01 12x	move vA, vB	Move the contents of one non-object register to another.
21 12x	array-length vA, vB	Store in the given destination register the length of the indicated array, in entries
28 10t	goto +AA	Unconditionally jump to the indicated instruction.

Table 1: Snippet of the opcode table for Dalvik bytecode[14]

Android runtime environment makes use of Dalvik bytecode, which is stored in .dex, or Dalvik Executable, files. These files contain compiled code that Android operating system runs. Originally Dalvik bytecode was designed for Dalvik Virtual Machine (DVM), which served as the original runtime for Android and was later replaced by ART. Despite that change Dalvik bytecode remains a staple in Android systems due to its optimization towards efficient use of limited resources, especially in smaller devices such as smartphones or tablets, and since this bytecode method remains largely unchanged ART is also backward-compatible.[15]

Android programs are distributed in packages, which amongst other many types of files also contain .dex files. By reverse-engineering these packages we can obtain desired .dex files for further processing. The tool used to achieve that will be Baksmali – a disassembler for the dex format used by dalvik, Android's Java VM implementation. [16] As of writing this thesis the version of the program is 2.5.2 and the syntax for the disassembler is:

baksmali disassemble app.apk -o app

The command will be used in this format as a prompt for Windows OS, called and looped, in case of processing multiple files, via a python script. While files obtained this way aren't of type .dex themselves but of .smali instead, they still provide a good overview as to the structure of original code and desired opcode sequences.

[13] D. W. Barron, *Assemblers and Loaders*, Macdonald & Co 1969
[14] https://source.android.com/docs/core/runtime/dalvik-bytecode
[15] https://source.android.com/docs/core/runtime
[16] https://github.com/JesusFreke/smali/wiki

Not all opcode instructions will be used in this project and a selection based on study detailing similar approach[17] will be chosen.

• *Move instructions (0x01 to 0x1C)*

• *Branch instructions (0x27 to 0x3D)*

• *Getter and setter instructions (0x44 to 0x6D)*

• *Method invoke instructions (0x6E to 0x78)*

• *Logic and arithmetic instructions (0x7B to 0xE2)*

This concludes the use of 28, 19, 42, 10 and 104 unique opcode instructions based on their respective type and totaling at a number of 203.

3.2 Data preprocessing

After getting access to application's code we can extract desired code snippets and use them as data for malware classification. In order to get these sequences we need to go through each .smali file and read needed opcode instructions into our program. To extract only these features a dictionary is constructed, for the program to compare read strings and further processing only those that meet the criteria.

Obtained strings are then placed into sequences of strings of uniform dimension, also known as n-grams. The *n* here is the dimension of each sequence. For the value of n equal to 2, for example, the resulting gram has a dimension of 2 which means each sequence contains two instructions. And in this case if we were to have a sentence "I really love programming." the resulting 2-grams would be [I, really]; [really, love]; [love, programming]. This makes it so that instead of analyzing every instance of on instruction being given what will be analyzed is the order in which these instructions appear.

In this work the values of *n* will be 2, 3 and 4, providing a substantial range for experimentation within the computational scope of this project, and to further reduce the dimensionality of data a feature hashing algorithm will be used.

Feature hashing is a vectorization technique utilized in order to efficiently reduce dimensionality of features. It works by turning arbitrary features into indexes in a vector or a matrix. When working with n-grams the number of features is equal to the number of unique n-grams, which can quickly get the number of features out of control, especially with higher values of *n*. Rather

[17] B. Kang, S. Y. Yerima, S. Sezer, K. Mclaughlin, N-gram Opcode Analysis for Android Malware Detection, International Journal on Cyber Situational Awareness 2016 p. 6

than dealing with thousands or even tens of thousands of features, with hashing the dimension can be reduced to a specified number.[18]

Instead of working with particular opcode sequences, these sequences can be assigned hash values and stored and ordered in buckets based on the hash value. In this way similar hashes are placed in the same bucket where their frequency can be counted. What that means is that what will actually be processed by the machine learning model is the frequency of hashed n-grams inside every bucket. While this method largely simplifies information, it also makes computation significantly easier.

The hashing algorithm used here will be MurmurHash3[19]. Mmh3is a non-cryptographic hash function, which means that it produces the same hash values for the same inputs. The non-cryptographic nature of this hashing method is essential for machine learning as it allows for making comparisons between same values, as opposed to proper cryptographic functions like SHA-2 which always produces unique values.

In addition to different *n* values there will also be multiple number of feature buckets used, namely: 256, 512, 1024 and 2048.

3.3 Dataset

The dataset consists of 1000 apks with an equal split between benign and malicious files. All the benign and most malicious files were obtained from publically available CIC-AndMal2017 dataset.[20] Other malicious files come from a GitHub repository.[21]

The file sizes between actual software and malware differ significantly. To combat any potential biases regarded to file size, or number of opcodes, bucket counts in each file will be divided by the total number of counted sequences, resulting in a dataset containing ratios of hashed values inside each bucket instead of absolute number of occurrences.

After feature extraction obtained data will be saved to a .pickle file for further processing and a .csv file for visualization.

3.4 Neural networks

Neural network (NN) is a type of machine learning algorithm inspired by the structure and function of biological neurons in real brains. Its main advantage is the ability to capture complex often hard to find relationships between pieces of data. Since our data is highly dimensional and encompasses different types of software, an NN seems like a good choice of architecture for our model.

---

[18] K. Weinberger, A. Dasgupta, J. Attenberg, J Langford, A. Smola, *Feature Hashing for Large Scale Multitask Learning*, Yahoo! Research, 2821 Mission College Blvd., Santa Clara, CA 95051 USA 2010

[19] https://pypi.org/project/mmh3/

[20] https://www.unb.ca/cic/datasets/andmal2017.html

[21] https://github.com/mstfknn/android-malware-sample-library

The input shape and in consequence the structure of the NN must be adjusted to match the number of features in data. Before deploying the model cross-validation, with 80/20 split, will be employed to test different datasets across 5 folds. This approach allows us to test the model comprehensively on different subsets of the data, ensuring robustness and reliability. The best performing model from cross-validation will be selected.

Finally, the chosen model will undergo training using all available data. This final training step aims to maximize the model's predictive capability by leveraging the entire dataset.

3.5 Used metrics

To evaluate the performance of a model multiple types of metrics will be employed.

Accuracy – a fundamental metric used to measure how well the model can accurately predict the correct class of instances in the dataset. There are to types of accuracy – the one used during training that represents the accuracy of predictions made during training and validation accuracy that measures the accuracy of predictions made on a validation set. Training accuracy grows as the model gets learns from supplied data and validation accuracy grows as the model gets better at prediction correct classes.

$$Accuracy = \frac{Number\ of\ correct\ predictions}{Total\ number\ of\ precitions}$$

Precision – is used to evaluate model's ability to correctly identify correct positive predictions. In binary classification positive prediction means that the model identifies the class as possessing tested features, so in case of this thesis of being a malware. True in this sense refers to making correct positive prediction i.e. file is truly malicious and false positive when the file is actually benign. This metric is important as it shows how likely the model is to flag benign files as being malicious and after being deployed in antimalware software raising unnecessary alarms.

$$Precision = \frac{True\ Positives}{True\ \ Positives + False\ Positives}$$

Recall – measures completeness of positive predictions made by a model. Compared to precision, recall would evaluate the model's performance in correctly detecting all positive instances. This metric is critical for malware detection as it represents the number of all correctly classified malicious files. When a model predicts that an instance is not malicious that instance is marked as negative. Similarly to positive predictions negative predictions can also be true and false. A false negative would mean that a model incorrectly classifies malware as being benign putting the user at risk.

$$Recall = \frac{True\ Positives}{True\ \ Positives + False\ Negatives}$$

F1 Score – serves as a balance between precision and recall. F1 helps to access classifier's performance based on two aforementioned metrics by summarizing model's ability to make correct positive predictions and capture all positive instances. F1 gives a harmonic mean, meaning the result gives more weight to lower values, making the score closer to lower of the two values and thus penalizing extreme values.

$$F1\ Score = 2 * \frac{Precision * Recall}{Precision + Recall}$$

ROC AUC (Receiver Operating Characteristic Area Under the Curve) – represents model's ability to distinguish between classes by using ROC curve to plot true positive rate against false positive rate across different threshold settings. AUC quantifies the overall performance based on that curve where higher values indicate more correct predictions. AUC values range from 0 to 1 with 1 being a perfect classifier and values under 0.5 indicating inverted classification.

Confusion Matrix – a graphical representation of predictions made across multiple instances in a dataset. Such matrix has two axes – the y axis representing actual class labels and the x axis representing predicted labels. It's yet another tool to help visualize model's ability to make correct predictions with the added benefit of seeing the actual number of all kinds of predictions made. It is particularly useful for validation datasets.

3.6 Data visualization

To visualize data a clustering algorithm can be used to showcase differences between the two classes of files. To achieve this we will use Weka[22] - an open-source machine learning software with plenty of useful data visualization tools. The clustering algorithm used will be Simple EM with default parameters.

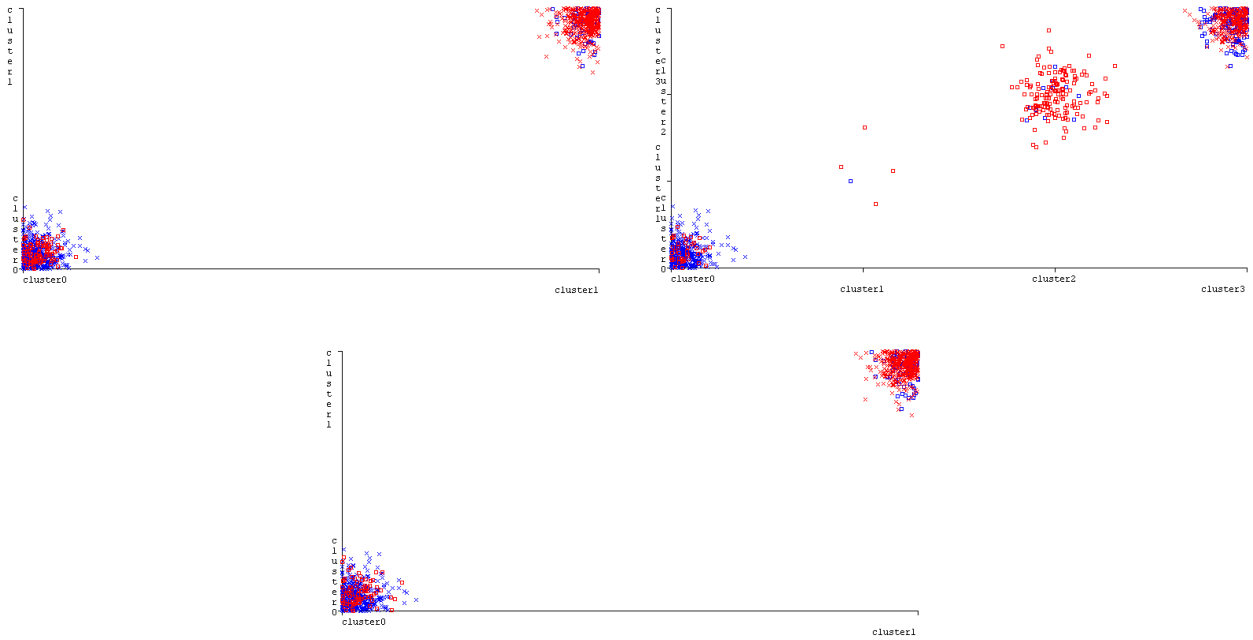
If there is enough variance between files there should be a proper number of clusters that separate benign and malicious files.

And as seen on provided graphs there is a enough of a distinction between samples for different clusters to be created. Blue and red represent benign and malicious files respectively. Out of all values of *n* the best results have been shown.

Out of all feature numbers the best results were achieved with 2048 for all values of *n*, with *n* equal to 2 and 3 providing best results and creating one cluster per class. For *n* value of 3, while there is a distinction between classes, the obtained clusters aren't as well pronounced.

Below these clusters are mapped onto graphs where benign samples are marked as blue and malicious one as red.

[22] https://ml.cms.waikato.ac.nz/weka/index.html



Picture 1: Visualization of clustered samples

The percentage of incorrectly clustered instances (left) and a number of created clusters (right) for all 12 combinations for *n* and feature number are described below.

features / n	2-grams		3-grams		4-grams	
256	48.4	4	60.5	10	56.8	8
512	50.1	5	43.0	5	47.1	7
1024	38.1	5	54.0	7	53.2	6
2048	18.7	2	35.8	4	18.2	2

Table 2: Results of data clustering

In conclusion, in seems like the accuracy grows with the number of features with 2-grams offering the most stable results. There also seems to be enough differences between samples to distinguish between two types of files and therefore obtained data can be used for malware detection.

The process of obtaining these features will be described in the next chapter.

4. Implementation

4.1 Goal of the project

The goal of this thesis is to create a machine learning model capable of differentiating between benign and malicious software within an acceptable level of accuracy. In order to accomplish it a dataset with different android apps has been collected. The necessary features will be extracted during implementation of the project with a description of this process. Different parameters of extracted features will be tested and compared to create the most accurate model possible. The method of classification chosen for this project is based on opcode frequencies and is thus limited to static signature analysis.

4.2 APK decompression

As mentioned earlier before any data can be extracted we need to extract proper files from android packages.

Before using this script a small change needs to be made in baksmali.bat file. The last line of that file asks for user input after executing a *disassemble* command. To achieve full automation this line ought to be deleted or commented out.

```
def decompress_apks(input_directory, output_directory):
    for root, dirs, files in os.walk(input_directory):
        for file_name in files:
            if file_name.endswith(".apk"):
                apk_path = os.path.join(root, file_name)
                              decompress_path   =   os.path.join(output_directory,
file_name.replace(".apk", "-decompressed"))

                try:
                    result = subprocess.run(
                                   ["java",  "-jar",  "C:\\Windows\\baksmali.jar",
"disassemble", apk_path, "-o", decompress_path],
                        capture_output=True,
                        text=True
                    )
                    if result.returncode != 0:
                           raise subprocess.CalledProcessError(result.returncode,
result.args, output=result.stdout, stderr=result.stderr)
                except subprocess.CalledProcessError as e:
                    with open("decompression_errors.log", "a") as log_file:
                                            log_file.write(f"Error   decompressing
{apk_path}:\n{e.stderr}\n")
    return 0
```

The script scans provided directory for .apk files and after finding one it issues a command to use baksmali on that file. In case of any errors there is a log file attached to record any unexpected instances. The second parameter passed to the *decompress* command is output directory where extracted files share the name with the original apk appended by “decompressed” word for further processing.

Despite baksmali being written in java it can still be invoked this way inside a python script ensuring easy integration and reuse in other parts of the application.

4.3 Feature extraction

After getting some .smali files the process of extracting features for training can be started. This process involved several steps and before it’s explanation, different parts of code used to achieve desired results will be shown.

Starting with - opcode extraction.

```
def extract_opcodes(smali_file):
    opcodes = []
    with open(smali_file, "r", encoding="utf-8") as f:
        for line in f:
            opcode_match = re.search(r'\s+([A-Za-z0-9_/-]+)', line)
            if opcode_match:
                opcode = opcode_match.group(1)
                      if  opcode  in  (move_instructions  |  branch_instructions  |
getter_setter_instructions         |         method_invoke_instructions         |
logic_arithmetic_instructions):
                    opcodes.append(opcode)
    return opcodes
```

This function is used to scan through a smali file to look for particular string - matching those described in an opcode dictionary encapsulating all mnemonics and values from ranges described in previous chapter.

After getting a list of opcodes, n-grams can be created.

```
def get_ngrams(directory, n):
    opcodes = []
    for root, dirs, files in os.walk(directory):
        for file in files:
            if file.endswith(".smali"):
                smali_file_path = os.path.join(root, file)
                opcodes.extend(extract_opcodes(smali_file_path))
    return list(ngrams(opcodes, n))
```

This function calls the previous function and after it's execution the list of opcode strings in converted into n-grams using NLTK library with opcode functionality.

Next we look at the hashing function.

```
def extract_features(sha, directory_path, hash_dim, n):
    if sha.endswith("-decompressed"):
        smali_file = os.path.join(directory_path, sha)
        tokens = get_ngrams(smali_file, n)
        token_hash_buckets = [(mmh3.hash(' '.join(w)) % (hash_dim - 1) + 1) for w
in tokens]
        token_buckets_counts = np.zeros(hash_dim)
        buckets, counts = np.unique(token_hash_buckets, return_counts=True)
        for bucket, count in zip(buckets, counts):
            token_buckets_counts[bucket] = count
        if "malware" in directory_path:
            malware = 1
        else:
            malware = 0
        return (sha, token_buckets_counts/np.sum(token_buckets_counts), malware)
```

Obtained n-grams are stored in a list as tokens and later hashed. From the result is then taken modulo *(hash_dim - 1)* and incremented by 1 to ensure that the hash values lie in the range [1, hash_dim-1]. This gives us a list where each element is a hash bucket index of an n-gram. Next a count array is initialized with a specific dimension that is filled with 0s. Later a number of all occurrences for each unique hash in calculated and that count is stored in corresponding index inside another list. After getting a list of bucket counts a class identifier is assigned based on directory path of files. Lastly counts in every bucket are divided by the total count of all opcode instances. The process is repeated for all files in specified input directory.

This function is used in training only and feature extraction for testing is done using different but similar function.

To efficiently invoke all these functions another one, enabling multi-processing, is used.

```
def extract_features_wrapper(file_path, directory_path, hash_dim, n):
    return extract_features(file_path, directory_path, hash_dim, n)

def    data_preprocessor(benign_files,    malicious_files,    path_to_benign_files,
path_to_malicious_files, features_length, n):
    print(f"Preprocessor working - n = {n}, Buckets = {features_length}")
    ben_features = Parallel(n_jobs=-1)(
```

```
                                                delayed(extract_features_wrapper)(sha,
directory_path=path_to_benign_files, hash_dim=features_length, n=n)
        for sha in benign_files
    )
    print("Benign files processed")
    print(f"Samples: {len(ben_features)} Buckets: {len(ben_features[0][1])}")
    mal_features = Parallel(n_jobs=-1)(
                                                delayed(extract_features_wrapper)(sha,
directory_path=path_to_malicious_files, hash_dim=features_length, n=n)
        for sha in malicious_files
    )
    print("Malicious files processed")
    print(f"Samples: {len(mal_features)} Buckets: {len(mal_features[0][1])}")
    all_features = ben_features + mal_features
     processed_data = [(sha, *counts, malware) for sha, counts, malware in
all_features]
    print("Data processed.")
    return processed_data
```

Before using the *Parallel* keyword a wrapper function is created to enable this functionality. The n_jobs parameter here has a value of -1, meaning that all processors will be used and is to be adjusted per user needs in case of too much overhead.

Finally the data can be exported.

```
def export_data(data, output_file, features_length):
    with open(output_file, 'w', newline='') as csvfile:
        label_row = "File Name",*[f"Bucket {i}" for i in range(features_length)]
,"Class"
        wr = csv.writer(csvfile)
        wr.writerow(label_row)
        for sample in data:
            wr.writerow(sample)
```

In this function before saving data to a .csv file an additional header row with a column for file name, dynamically determined number of bucket columns and an identifier column is added for readability.

4.4 Model structure

With prepared data the training of a model can start. As mentioned earlier model architecture must be altered to fit a desired number of features and a function doing that is implemented. More features mean the architecture needs to be more complex.

```
def pick_model(shape):
```

```
    model = Sequential()
    if (shape==2048):
        model.add(Dense(units=1024, input_shape=(2048,)))
        model.add(Dense(units=512))
        model.add(Dense(units=256))
        model.add(Dense(units=64))
        model.add(Dense(units=1, activation='sigmoid'))
    elif (shape==1024):
        model.add(Dense(units=512, input_shape=(1024,)))
        model.add(Dense(units=256))
        model.add(Dense(units=64))
        model.add(Dense(units=1, activation='sigmoid'))
    elif (shape==512):
        model.add(Dense(units=256, input_shape=(512,)))
        model.add(Dense(units=64))
        model.add(Dense(units=1, activation='sigmoid'))
    elif (shape==256):
        model.add(Dense(units=128, input_shape=(256,)))
        model.add(Dense(units=64))
        model.add(Dense(units=1, activation='sigmoid'))
    return model
```

While the input layer varies the last output layer has uniform dimension of 1 across all architectures. That single neuron decides between two classes by producing a value between 0 and 1. Predicted class is based on proximity of the resulting number to one of the extremes.

The activation function here is *sigmoid.* It's a widely used function for binary classification tasks and helps in calculating that desired range for prediction value.

To ensure an even split between malware and benign samples in each fold a variant of k-fold cross-validation from *scikit-learn* library called *StratifiedKFold* is used. This ensures that data distribution in each fold is more balanced and more representative of the entire dataset.

```
kf = StratifiedKFold (5, shuffle=True, random_state=42)
```

The shuffle parameter is used to randomly shuffle data before splitting helping in creating more randomized and diverse fold. The other parameter acts as a seed to ensure reproducibility between the folds applying the same shuffling and splitting every time the code is executed, allowing for consistent and comparable results.

```
fold = 0
for train, test in kf.split(x, y):
    fold+=1
    print(f"Fold: {fold}")
```

```
    x_train = x[train]
    x_test = x[test]
    y_train = y[train]
    y_test = y[test]

    model = pick_model(features_length)
                model.compile(optimizer="adam",       loss="binary_crossentropy",
metrics=["accuracy"])
        history   =   model.fit(x_train,   y_train,   epochs=20,   batch_size=100,
validation_data=(x_test, y_test), shuffle=True, verbose=0)
```

After creating a model object that model can be compiled and trained. When compiling a model numerous parameters can be entered. In this model the first parameter is the optimizer that specifies the algorithm used to update weights during training. The optimizer chosen here is *adam*, short for Adaptive Moment Estimation. *Adam* is useful because of its ability to maintain separate learning rates and updates for each parameter. It's also offers efficient convergence in high dimensional spaces and is resistant to a lot of noisy data.

Another parameter is the loss function responsible for calculating how well the model is adjusting to data by calculating the difference between predicted and actual values. Binary crossentropy is a standard and effective choice for binary classification tasks and is also used here.

Last parameter represents the accuracy of predicted values giving a generalized and simple metric to monitor during training.

To record the training process a *history* object is created before training. This method allows computing changes in metrics during different epochs of training and makes visualization of training process using graphs simpler.

Data is split into training and validation subsets for each epoch. Choice of the number of epochs and batches was arbitrarily decided during experimentation and model tuning based on produced results. Chosen number of epochs gives the model time to converge without resulting in too much overfitting. Models also seemed to do better with larger batches, which might be because of the diverse nature of software in the dataset. The size is also not too large as to help the model generalize better.

5. Test of solution

5.1 Model training

The training phase begins with the selected model architectures. In this section, we will discuss the results obtained with various parameter settings, which are documented in the attached directory.

Based on cross-validation for all tested n-grams the best number of features based on average performance across folds was the highest one – 2048. Despite higher initial dimensionality spacing extracted opcode sequences into more buckets yielded superior results compared to lower values, in fact the lower the number of features the worse the performance. This indicates that more diverse and numerous features might be better for such tasks.

Detailed results from these tests are compiled in a table.

n = 2	avg	max	min	delta
Accuracy	0.907	0.92	0.88	0.04
Precision	0.953158	1	0.925532	0.074468
Recall	0.858	0.9	0.76	0.14
F1-Score	0.901499	0.918367	0.863636	0.054731
ROC AUC	0.97024	0.9811	0.9537	0.0274

n = 3	avg	max	min	delta
Accuracy	0.913	0.935	0.89	0.045
Precision	0.926095	0.977273	0.857143	0.12013
Recall	0.902	0.96	0.86	0.1
F1-Score	0.912203	0.934673	0.89	0.044673
ROC AUC	0.97303	0.9856	0.947	0.0386

n = 4	avg	max	min	delta
Accuracy	0.902	0.945	0.82	0.125
Precision	0.893564	0.949495	0.742424	0.207071
Recall	0.926	0.98	0.83	0.15
F1-Score	0.905858	0.944724	0.844828	0.099896
ROC AUC	0.97038	0.9897	0.9549	0.0348

Table 3: Comparison of best results from cross-validation

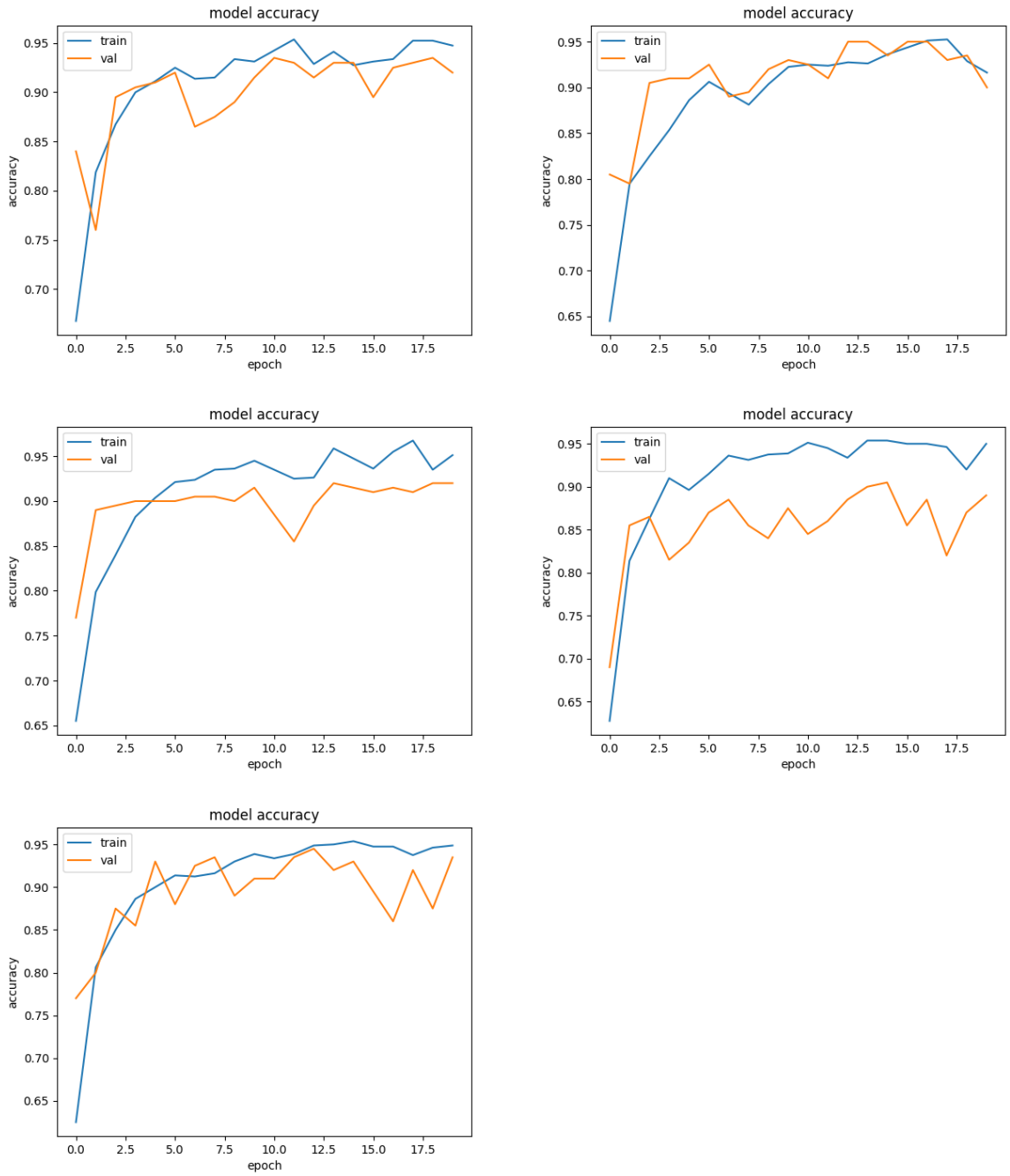
The lowest *n* performed the best in terms of precision on average, making it best at correctly classifying benign software. It was also the worst at detecting malware based on the lowest value of recall and an overall most unbalanced F1 score out of the three variants. Despite this imbalance, it showed the smallest variation (delta) across different training sessions, suggesting greater stability in model performance.

Value of 3 produced the best overall results from all variants. The highest F1 and AUC indicate a healthy balance between making false predictions and thus effectively flagging malicious software without producing excessive alarms for benign programs. It was also the most accurate with an impressive rate of 91%. It also has the lowest variance in terms of recall making it the most stable in that vital regard.

The last variant proved to be the most unstable, making the final model architecture highly varied and unpredictable across different training sessions. It, however, was the best at correctly flagging malware with the highest recall. That would technically result in the safest model; however the unpredictability indicated by testing might not necessarily reproduce these results.

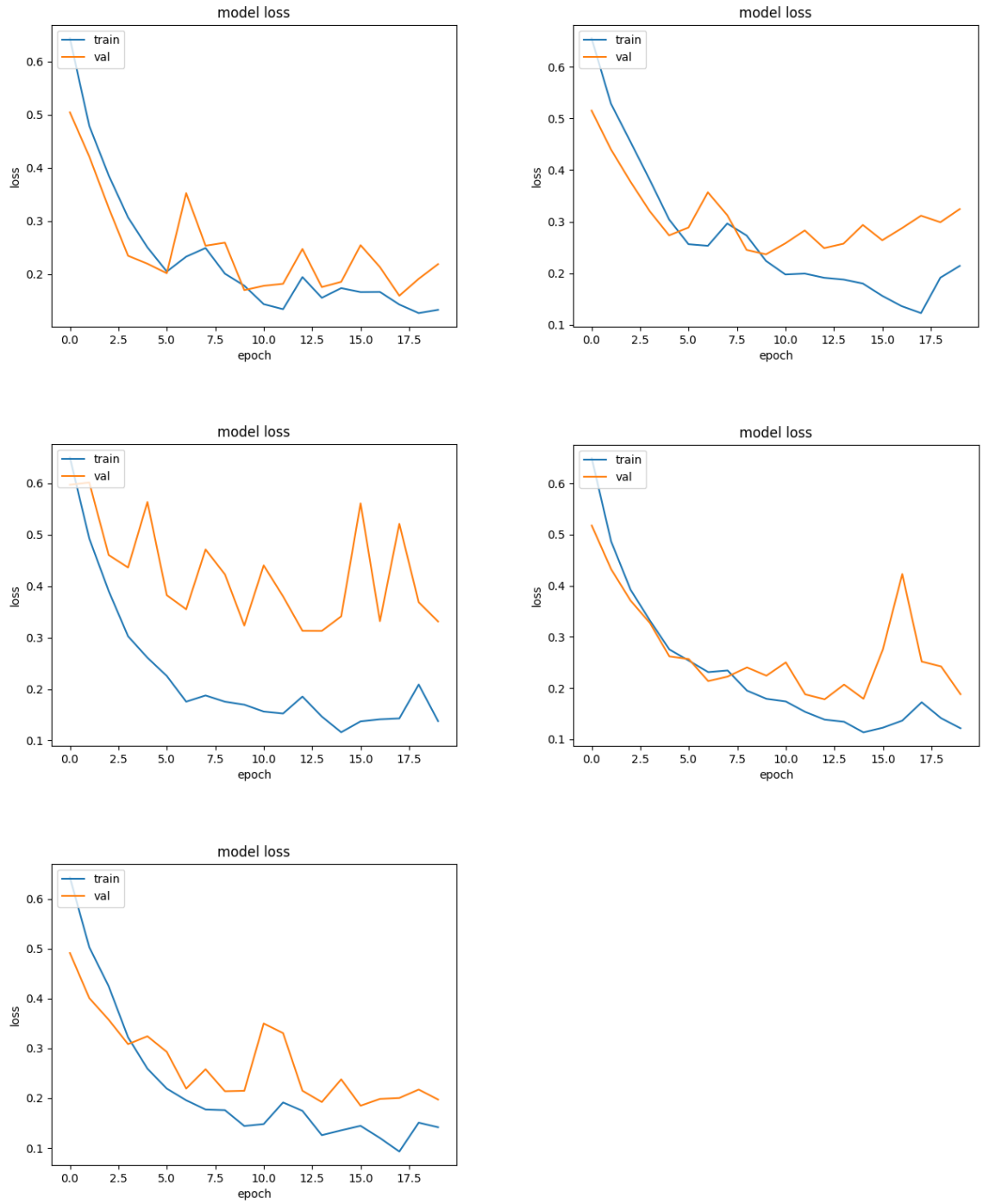
Since $n = 3$ with buckets numbering 2048 proved to be the best combination a more detailed overview of experimentation with these parameters shall be presented.

First - the accuracy across folds, followed by loss and ROC.



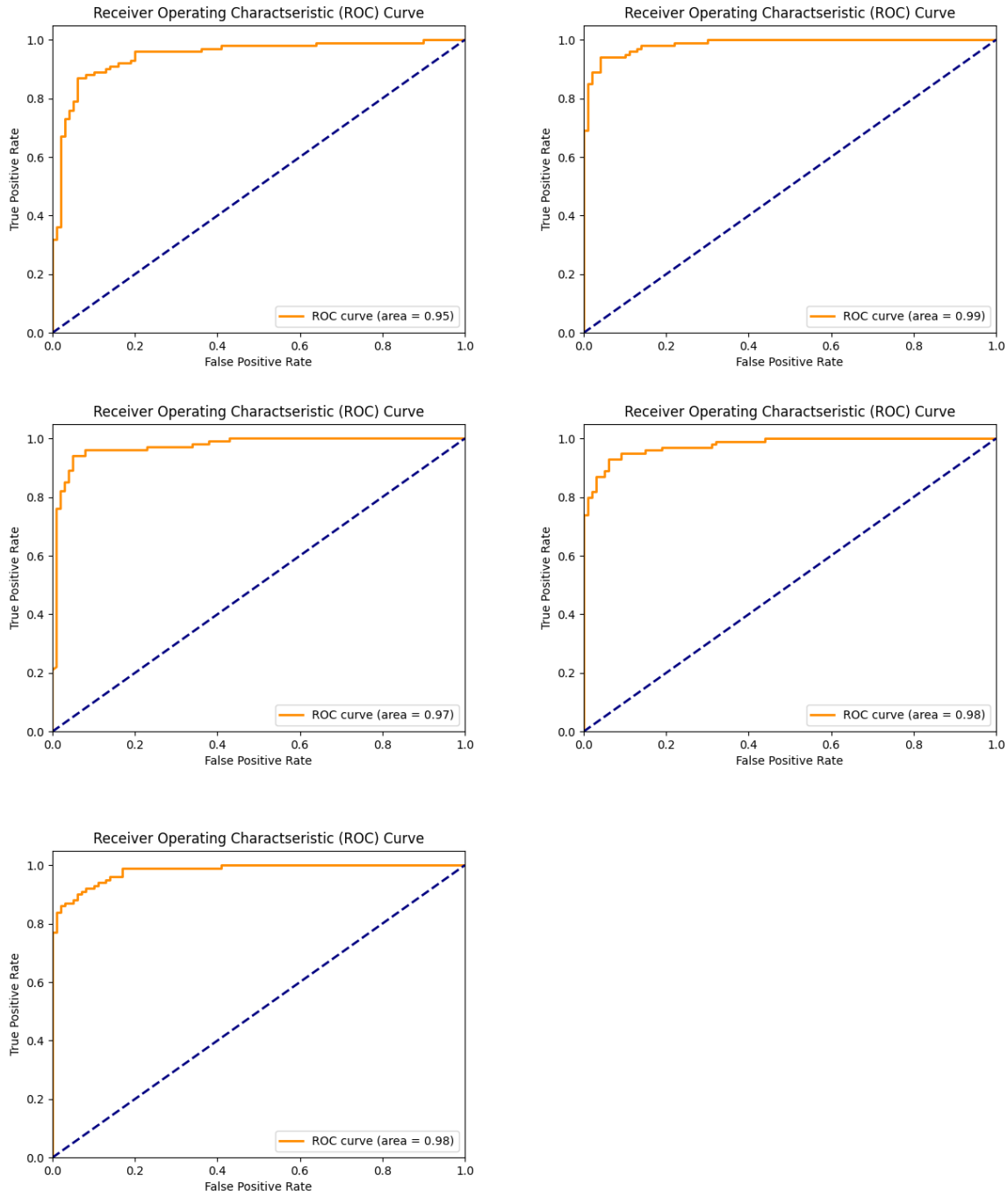
Picture 2: Accuracy across folds

As seen on the graphs the model managed to converge rather well across different folds reaching top performance at around the fifth epoch. The curve displaying validation accuracy, however, isn't as smooth as it would be desired suggesting potential challenges with model's generalization capabilities.



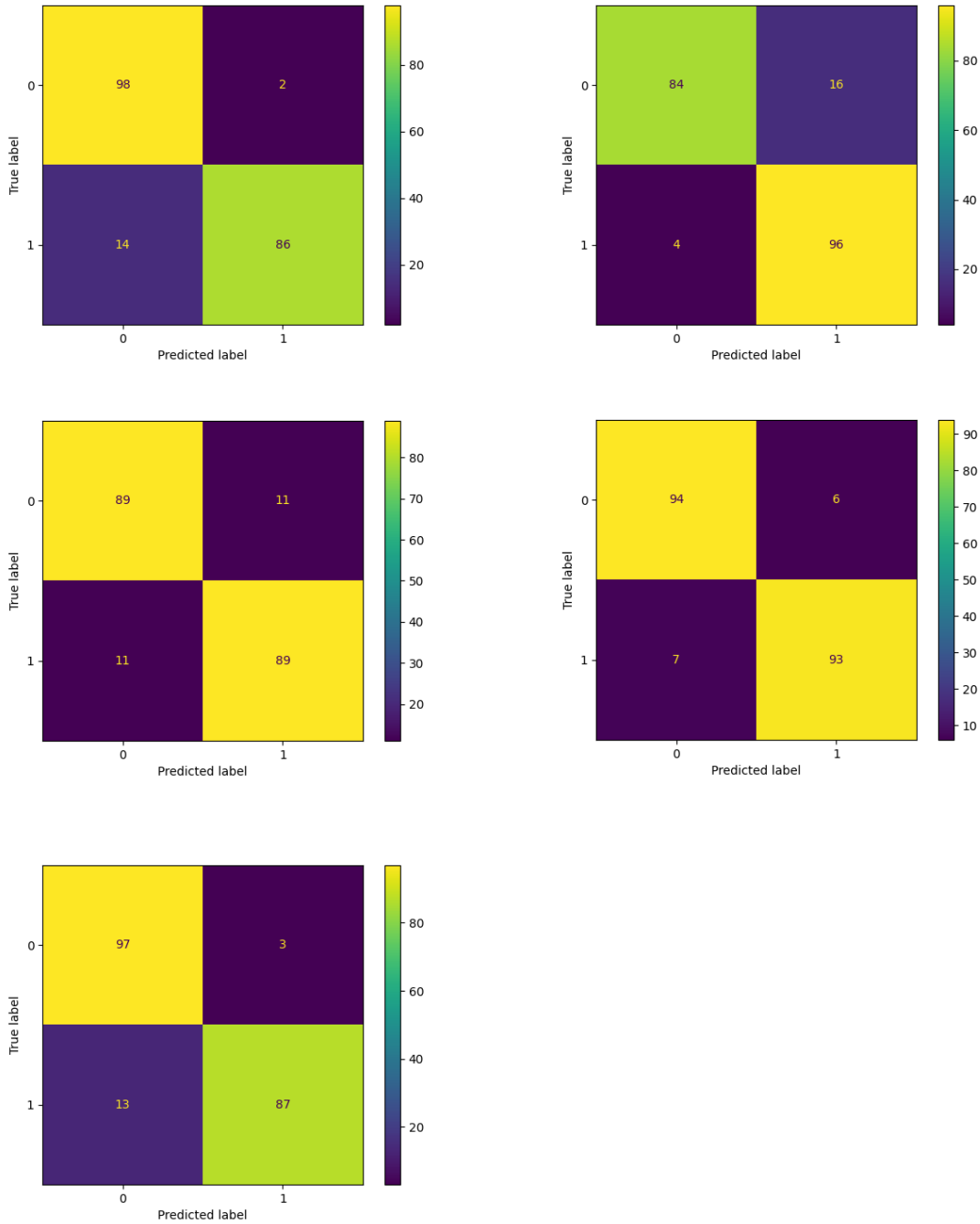
Picture 3: Loss across folds

The loss isn't as uniform across fold as accuracy was but the convergence can still be seen. The higher value of validation loss might indicate potential overfitting and while it gets lower in general with further epochs it also diverges more from training loss.



Picture 4: ROC AUC across folds

The AUC was consistently very high in each fold indicating a strong confidence in the model's predictive ability. Based on these curves the threshold of 0.8 seems to be producing the lowest number of false positives while retaining a high rate of true positives. In most fold the threshold as high as 0.9 still results in false positive rate of under 0.1. The ROC analysis provides a very promising estimation of model's capabilities.



Picture 5: Confusin matrices across folds

The accuracy of predictions displayed in confusion matrices varies but shows ability to differentiate between classes at satisfactory level. This variance can be explained by small size of the dataset and high variance between samples.

Interestingly, with these parameters the clustering done in the previous chapter had the worse outcome. It could be attributed to the fact that 3-grams helped to catch patterns not present or as

pronounced as in the other grams. The clustering algorithm assigns one class to one cluster, so having more clusters results in less recorded accuracy. Having more cluster might mean that there are more differentiating patterns between files of the same class.

5.2 Model deployment

After deciding on feature parameters the final model trained on the entire dataset can be created. It shall inherit the same structure as the model during testing but without splitting and is saved as a .keras file.

While developing a complete malware detection system is beyond the scope of this thesis, a demonstration app can be created to showcase its core functionalities.

Before using the model we need to ensure that input data has to have the same shape as the one used in training so a new function extracting features from a single file without labels needs to be added.

```
def extract_features_test(directory_path, hash_dim, n):
    smali_file = directory_path
    tokens = get_ngrams(smali_file, n)
    token_hash_buckets = [(mmh3.hash(' '.join(w)) % (hash_dim - 1) + 1) for w in
tokens]
    token_buckets_counts = np.zeros(hash_dim)
    buckets, counts = np.unique(token_hash_buckets, return_counts=True)
    for bucket, count in zip(buckets, counts):
        token_buckets_counts[bucket] = count
    return (token_buckets_counts/np.sum(token_buckets_counts))
```

This function returns a feature vector for a single sample that can be fed into the model and is invoked using a wrapper function.

```
def process_file(file_path):
    try:
        decompress_apk(file_path, temp_path)
        data = np.array(extract_features_test(temp_path, 2048, 3))
        prediction = model.predict(data.reshape(1, -1))
        if prediction > 0.8:
            label.config(text=f"Confidence = {prediction}\n!!!MALWARE!!!",
fg="red")
        else:
            label.config(text=f"Confidence = {prediction}\nBenign", fg="green")
    except Exception as e:
        label.config(text=f"Error: {str(e)}", fg="red")
    finally:
        shutil.rmtree(temp_path)
```

```
        browse_button.config(state=tk.NORMAL)
```

Said function before calling the feature extractor calls the file decompressor to obtain .smali files needed for analysis. Since the model was trained on multi-dimensional vectors the data before being processed by the model needs to be reshaped. The arguments for .reshape function here mean that the array has one row and a number of columns automatically based on original size. This changes the array shape from (2048,) to (1, 2048) resulting in a 2D array with one row and 2048 feature columns instead of an array of 2048 features represented as rows.

The value returned by model prediction is a float number between 0 and 1. Based on the test results the threshold here is set to 0.8 meaning that any confidence value equal or greater than this value is classified as malicious.

```
def browse_file():
    label.config(text="")
    browse_button.config(state=tk.DISABLED)
    file_path = filedialog.askopenfilename(filetypes=[("APK files", "*.apk")])
    if file_path.endswith(".apk"):
        threading.Thread(target=process_file, args=(file_path,)).start()
    else:
        label.config(text="Invalid file type selected.", fg="red")
        browse_button.config(state=tk.NORMAL)
```

Above function is called upon pressing a button and allows for selecting a file to be scanned. The threading is employed to ensure that the GUI remains responsive in case of potentially time-consuming operations, especially when processing larger files. It also ensures the program won't freeze during executing all these functions.

6. Conclusions

With the goal of creating a malware classifier using NN the chosen method of opcode n-gram frequency counting proved to be largely successful. Based on model testing using cross-validation the training results were similar across different folds, indicating a general ability of a model to differentiate between classes in the provided dataset.

From performed experiments it appeared that the network worked better with more features (2048) and a midrange length of n-grams (3). This result aligns with findings described in a referenced paper, which demonstrated optimal performance with *n* values of 3 and 4[23] and another paper showing high performance with 3-grams.[24] A higher number of features create a more detailed and robust view of the specifications of each tested file. The value of n chosen for the final model strikes a balance where extracted opcode sequences are varied enough to map out differences between files without creating excessive variation that could diminish the effectiveness of comparisons.

Despite the hashing method used the data ended up being workable despite a drastic reduction in dimensionality and simplification. In contrast to using actual n-gram strings the calculation of opcode sequence occurrences and mapping them out in a vector is way more efficient both in terms of processing power but also in terms memory.

Across training folds most metrics stayed consistent with ROC showing a solid 0.1 FPR at thresholds higher than 0.8 with AUC not falling below 0.95. However, validation accuracy and loss exhibited significant oscillations at times, indicating potential weaknesses in the model's ability to adapt to new information. This behavior may be attributed to the relatively small dataset size, comprising only 1000 samples.

The approach is limited to static analysis, making it too simplistic to be effective in real-world scenarios on its own. Future work should consider integrating dynamic analysis techniques to enhance the robustness and applicability of the malware detection system.

[23] B. Kang, S. Y. Yerima, S. Sezer, K. Mclaughlin, op. cit. p. 18-19

[24] P. N. Yeboah, S. K. Amuquandoh, H. B. B. Musah, Malware Detection Using Ensemble N-gram Opcode Sequences, International Journal of Interactive Mobile Technologies (iJIM) 2021, p. 27-29

7. Bibliography

Literature:

1. ENISA, ENISA THREAT LANDSCAPE, October 2023
2. N. Idika, A. Mathur, A survey of malware detection techniques, Purude Univeristy 2007
3. R. Sihwail, K. Omar, K. A. Z. Ariffin, A Survey on Malware Analysis Techniques: Static, Dynamic, Hybrid and Memory Analysis, International Journal on Advanced Science Engineering and Information Technology 8(4-2):1662, 2018
4. Android Security Paper 2023
5. Ö.Aslan, R.Samet, A Comprehensive Review on Malware Detection Approaches, IEEE Access, January 2020
6. J. Saxe, H. Sanders, Malware Data Science: Attack Detection and Attribution, No Starch Press 2018
7. D. W. Barron, Assemblers and Loaders, Macdonald & Co 1969
8. B. Kang, S. Y. Yerima, S. Sezer, K. Mclaughlin, N-gram Opcode Analysis for Android Malware Detection, International Journal on Cyber Situational Awareness 2016
9. K. Weinberger, A. Dasgupta, J. Attenberg, J Langford, A. Smola, Feature Hashing for Large Scale Multitask Learning, Yahoo! Research, 2821 Mission College Blvd., Santa Clara, CA 95051 USA 2010
10. P. N. Yeboah, S. K. Amuquandoh, H. B. B. Musah, Malware Detection Using Ensemble N-gram Opcode Sequences, International Journal of Interactive Mobile Technologies (iJIM) 2021

Sources:

1. https://www.ibm.com/blog/malware-history/
2. https://developers.google.com/android/play-protect
3. https://source.android.com/docs/core/runtime
4. https://github.com/JesusFreke/smali/wiki
5. https://pypi.org/project/mmh3/
6. https://www.unb.ca/cic/datasets/andmal2017.html
7. https://github.com/mstfknn/android-malware-sample-library
8. https://ml.cms.waikato.ac.nz/weka/index.html

8. List of figures and tables

Figures:

1. Visualization of clustered samples
2. Accuracy across folds
3. Loss across folds
4. ROC AUC across folds
5. Confusion matrices across folds

Tables:

1. Snippet of the opcode table for Dalvik bytecode
2. Results of data clustering
3. Comparison of best results from cross-validation